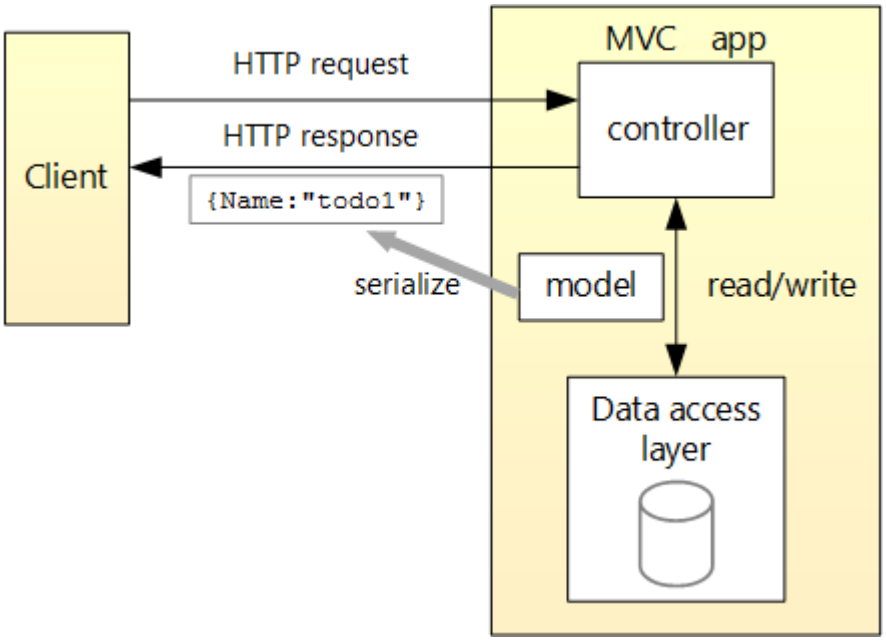
Разработка Web API на ASP.NET Core

Разработайте простой Web-API для управления списком задач.

Обзор
Целью создания API является создание удобного способа получения и изменения данных, например списка задач. Вот эндпоинты которые необходимо создать:

Запрос	Описание	Тело запроса	Тело ответа
GET /api/todo	Получение всех элементов	Нет	Список задач
GET /api/todo /{id}	Получить элемент по идентификатору	Нет	Определенная задача
POST /api/todo	Добавить новый элемент	Новая задача	Добавленная задача
PUT /api/todo/{id}	Обновить элемент	Задача	Нет
PATCH /api/todo /{id}	Обновить элемент	Задача	Нет
DELETE /api/todo /{id}	Удалить элемент	Нет	Нет

На следующей диаграмме показана архитектура приложения.

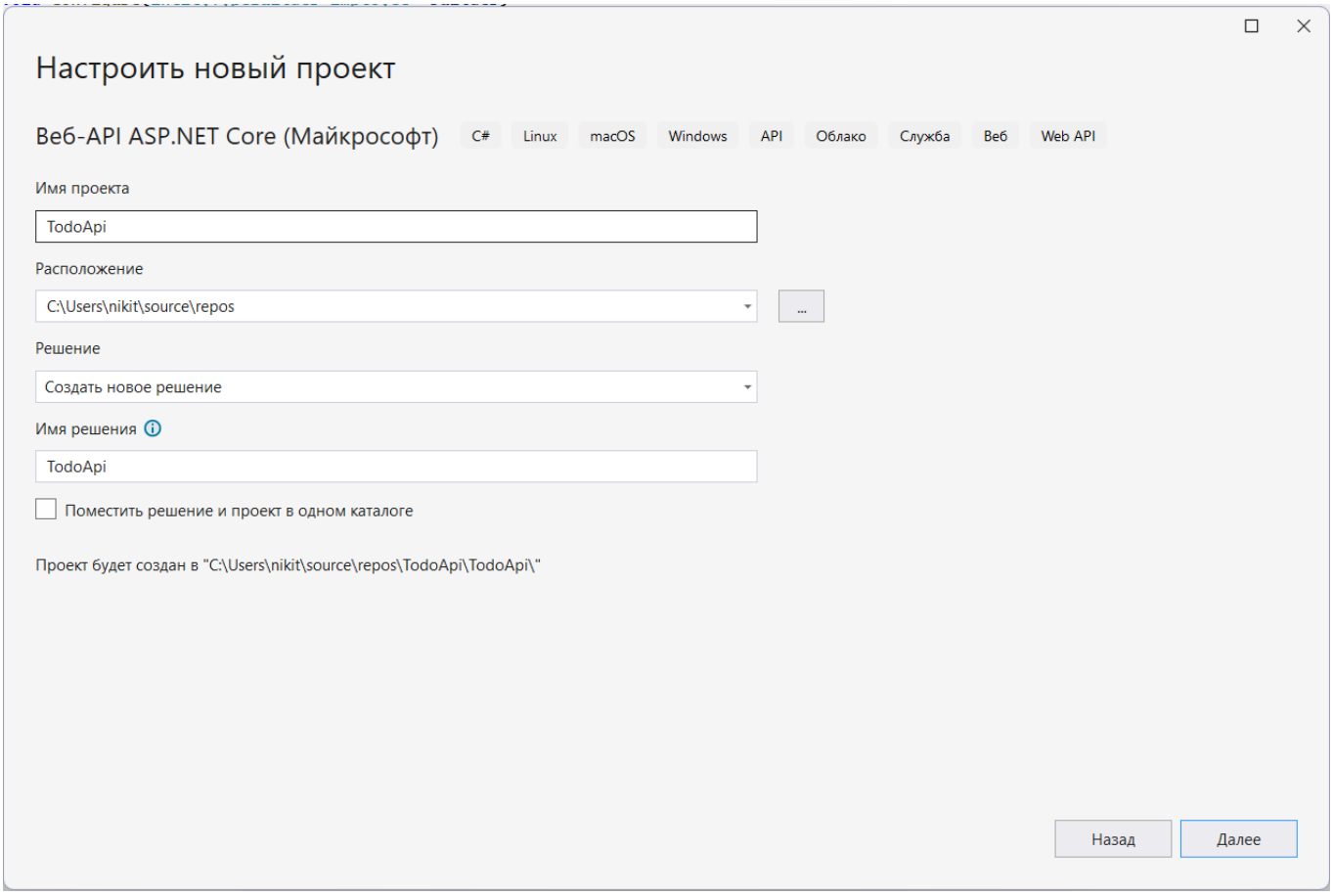


- **Клиент** - это все, что может использовать ваше API (браузер, мобильное приложение и т.д.). В этом руководстве вы не разрабатываете клиентскую часть. Вы будете использовать сервис Postman для тестирования приложения;
- **Модель** - это объект, представляющий данные в вашем приложении. В этом случае единственной моделью является элемент списка задач. Модели представлены в виде простых классов C#;
- **Контроллер** - это объект, который обрабатывает HTTP-запросы и создает HTTP-ответы. В данном примере приложение, для простоты, вы не будете использовать базу данных. Но вы все равно включите уровень доступа к данным к приложению, чтобы проиллюстрировать разделение между API и уровнем данных.

Создание проекта

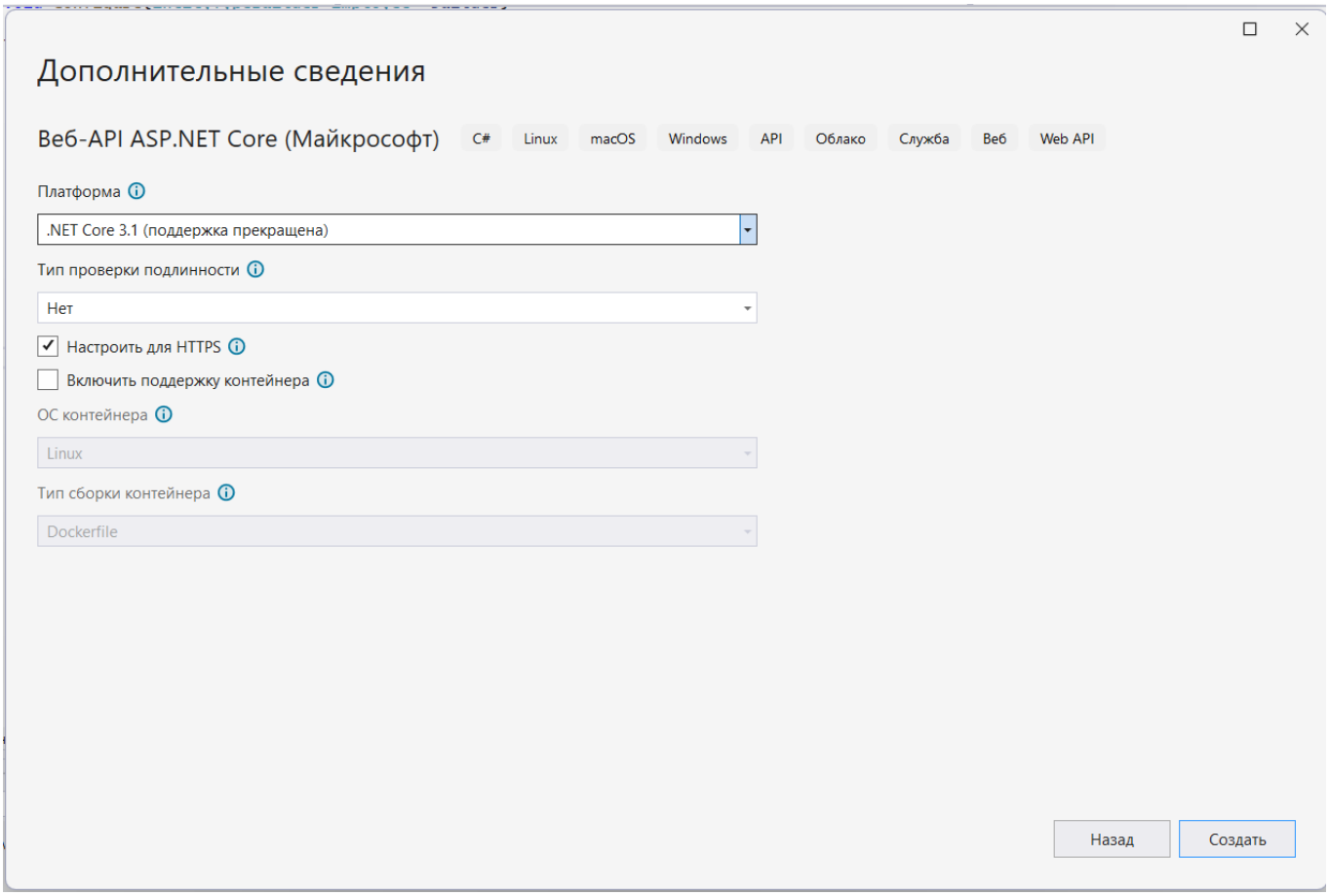
Запустите Visual Studio. В меню **Файл** выберите **Создать** > **Проект**.

Выберите **Веб-API ASP.NET Core**. Назовите проект `TodoApi` и нажмите **Далее**.



Выберите версию платформы для работы (на компьютерах скорее всего установлено .net

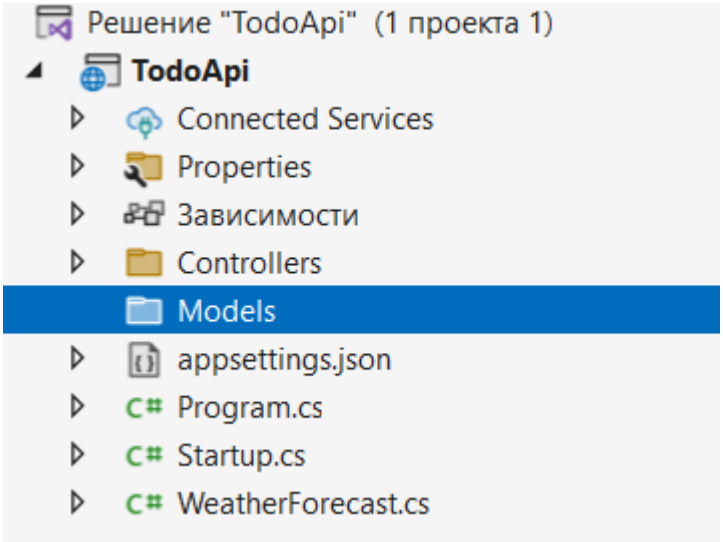
core 3)



Добавьте класс-модель

Модель - это объект, представляющий данные в вашем приложении. В этом случае единственной моделью является элемент списка задача.

Добавьте новую папку по названием "Models". В Обозревателе решений щелкните проект правой кнопкой мыши. Выберите **Добавить** > **Новая папка**. Назовите папку **Models**.



Добавьте класс **TodoItem**. Щелкните правой кнопкой мыши по папке **Models** и выберите **Добавить** > **Класс**. Назовите класс **TodoItem** и нажмите **Добавить**.
Замените сгенерированный код на:

```
namespace TodoApi.Models
{
    public class TodoItem
    {
        public string Key { get; set; }
        public string Name { get; set; }
        public bool IsComplete { get; set; }
    }
}
```

Данные свойства определяют значения которые можно будет расположить в каждой строчке-объекте

Добавьте класс репозитория

Репозиторий - это объект, который инкапсулирует уровень данных и содержит логику для извлечения данных и сопоставления их с моделью. Хотя в примере приложения не используется база данных, полезно посмотреть, как вы можете внедрить репозиторий в свои контроллеры. Создайте код репозитория в папке **Models**.

Начните с определения интерфейса с именем **ITodoRepository**. Используйте шаблон класса (**Добавить новый элемент** > **Класс**).

```
using System.Collections.Generic;

namespace TodoApi.Models
{
    public interface ITodoRepository
    {
        void Add(TodoItem item);
        IEnumerable<TodoItem> GetAll();
        TodoItem Find(string key);
        TodoItem Remove(string key);
        void Update(TodoItem item);
    }
}
```

Этот интерфейс определяет основные операции CRUD.

Затем добавьте **TodoRepository** класс, реализующий **ITodoRepository**:

```
using System;
using System.Collections.Generic;
using System.Collections.Concurrent;

namespace TodoApi.Models
{
    public class TodoRepository : ITodoRepository
    {
        private static ConcurrentDictionary<string, TodoItem> _todos =
              new ConcurrentDictionary<string, TodoItem>();

        public TodoRepository()
        {
            Add(new TodoItem { Name = "Item1" });
        }

        public IEnumerable<TodoItem> GetAll()
        {
            return _todos.Values;
        }

        public void Add(TodoItem item)
        {
            item.Key = Guid.NewGuid().ToString();
            _todos[item.Key] = item;
        }

        public TodoItem Find(string key)
        {
            TodoItem item;
            _todos.TryGetValue(key, out item);
            return item;
        }

        public TodoItem Remove(string key)
        {
            TodoItem item;
            _todos.TryRemove(key, out item);
            return item;
        }

        public void Update(TodoItem item)
        {
            _todos[item.Key] = item;
        }
```

```
    }
}
```

Соберите приложение, чтобы убедиться, что у вас нет ошибок компилятора.

Зарегистрируйте репозиторий

Определив интерфейс репозитория, мы можем отделить класс репозитория от контроллера MVC, который его использует. Вместо создания экземпляра **TodoRepository** внутри контроллера внедрите **ITodoRepository**, используя встроенную инструменты в ASP.NET Core для внедрения зависимостей.

Такой подход упрощает модульное тестирование ваших контроллеров. Модульные тесты должны включать заглушку **ITodoRepository**. Таким образом, тест нацелен исключительно на логику контроллера, а не на уровень доступа к данным.

Чтобы внедрить репозиторий в контроллер, нам нужно зарегистрировать его в контейнере DI. Откройте файл **Startup.cs**. Добавьте следующую директиву using:

```
using TodoApi.Models;
```

В методе **ConfigureServices** добавьте следующий программный код:

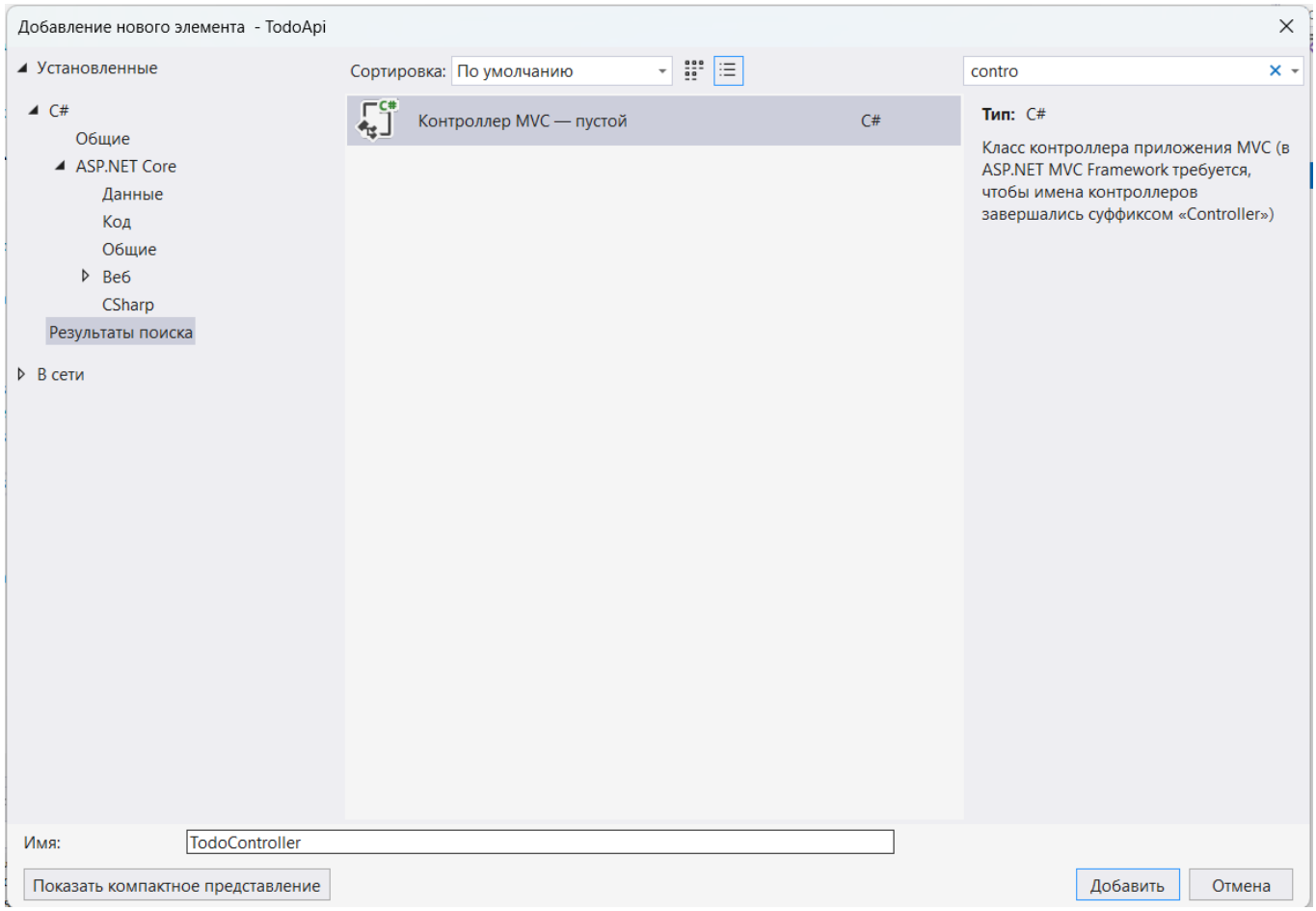
```
public void ConfigureServices(IServiceCollection services)
{
        // Сгенерированный код выше
        services.AddMvc();

        services.AddSingleton<ITodoRepository, TodoRepository>();
}
```

Добавление контроллера

В обозревателе решений щелкните правой кнопкой мыши папку **Controllers**. Выберите **Добавить** > **Новый элемент**. В диалоговом окне **Добавить новый элемент** выберите

шаблон **Контроллер MVC - пустой**. Назовите класс **TodoController**.



Замените сгенерированный код:

```
using System.Collections.Generic;
using Microsoft.AspNetCore.Mvc;
using TodoApi.Models;

namespace TodoApi.Controllers
{
    [Route("api/[controller]")]
    public class TodoController : Controller
    {
        public TodoController(ITodoRepository todoItems)
        {
            TodoItems = todoItems;
        }
        public ITodoRepository TodoItems { get; set; }
    }
}
```

Это определяет пустой класс контроллера.

Получение элементов списка задач

Чтобы получить элементы списка задач, добавьте следующие методы в **TodoController** класс.

```
[HttpGet]
public IEnumerable<TodoItem> GetAll()
{
    return TodoItems.GetAll();
}

[HttpGet("{id}", Name = "GetTodo")]
public IActionResult GetById(string id)
{
    var item = TodoItems.Find(id);
    if (item == null)
    {
        return NotFound();
    }
    return new ObjectResult(item);
}
```

Эти методы реализуют два метода GET:

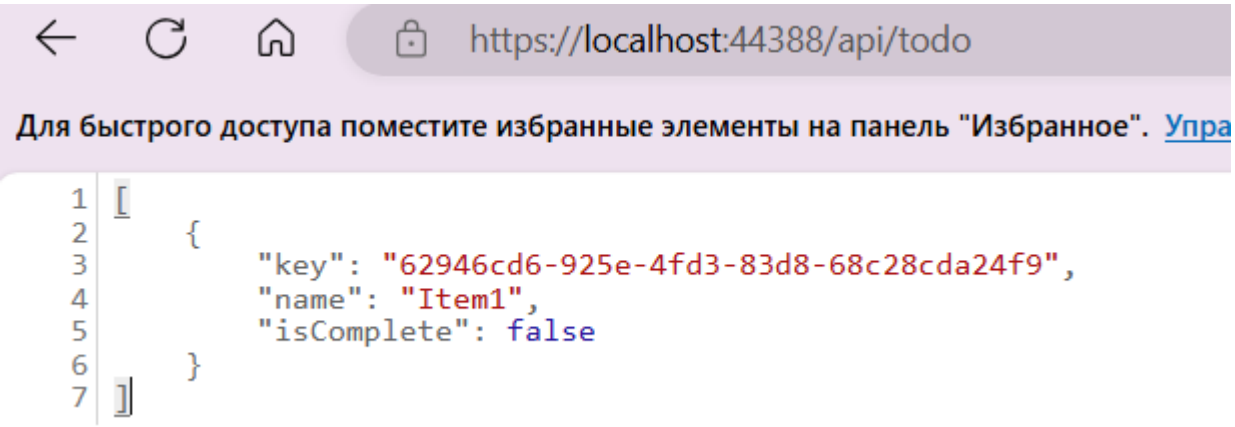
- GET /api/todo
- GET /api/todo/{id}

Вот пример HTTP-ответа **GetAll** метода:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Server: Microsoft-IIS/10.0
Date: Thu, 18 Jun 2015 20:51:10 GMT
Content-Length: 82
```

Вы можете проверить методы с помощью любого браузера. Соберите и запустите приложение, будет открыта страница по пути по умолчанию. Измените например, на метод получения всех задач, дописав к локальному расположению запущенного Веб-API

эндпоинты /api/todo.



Разберем некоторые моменты в программном коде **TodoController**

Маршрутизация и URL-адреса

Атрибут **[HttpGet]** определяет метод HTTP GET. URL-путь для каждого метода строится следующим образом:

- За шаблон берется строка в атрибуте **Route** всего контроллера: **[Route("api/[controller]")]**
- Часть **[controller]** заменяется именем контроллера, которое в свою очередь, является именем класса контроллера за вычетом суффикса “Controller”. В этом примере имя класса контроллера - **Todo**Controller, а корневое имя - “todo”, не учитывая регистр.

  https://localhost:44388/api/todo

- Если у метода внутри контроллера у атрибута **[HttpGet]** есть свой шаблон, добавьте ее к пути. В примере строка шаблона не используется.

Например, в методе **GetById** методе:

```
[HttpGet("{id}", Name = "GetTodo")]
public IActionResult GetById(string id)
```

- **"{id}"** это переменная-заполнитель для уникального идентификатора элемента. При вызове метода **GetById** он присваивает значение “{id}” в URL-адресе параметру id в методе.

Name = "GetTodo" создает именованный маршрут и позволяет вам ссылаться на этот маршрут в HTTP-ответе.

Возвращаемые значения

Метод **GetAll** сериализует объект который необходимо отправить в формат JSON и записывает JSON в тело ответа. В зависимости от успешности обработки данных метод **GetAll** возвращает:

- Если ни один элемент не соответствует запрошенному идентификатору, метод возвращает ошибку 404.
- В ином случае метод возвращает код 200 с телом в формате JSON

Запуск приложения

В Visual Studio нажмите CTRL + F5, чтобы запустить приложение. Visual Studio запускает браузер и переходит к `http://localhost:port/api/todo`, где *port* - это случайно выбранный номер порта.

Реализуйте другие операции CRUD

Добавьте методы **Create**, **Update** и **Delete** в контроллер. Создайте новые методы в класс **TodoController** после методов добавленных вами ранее.

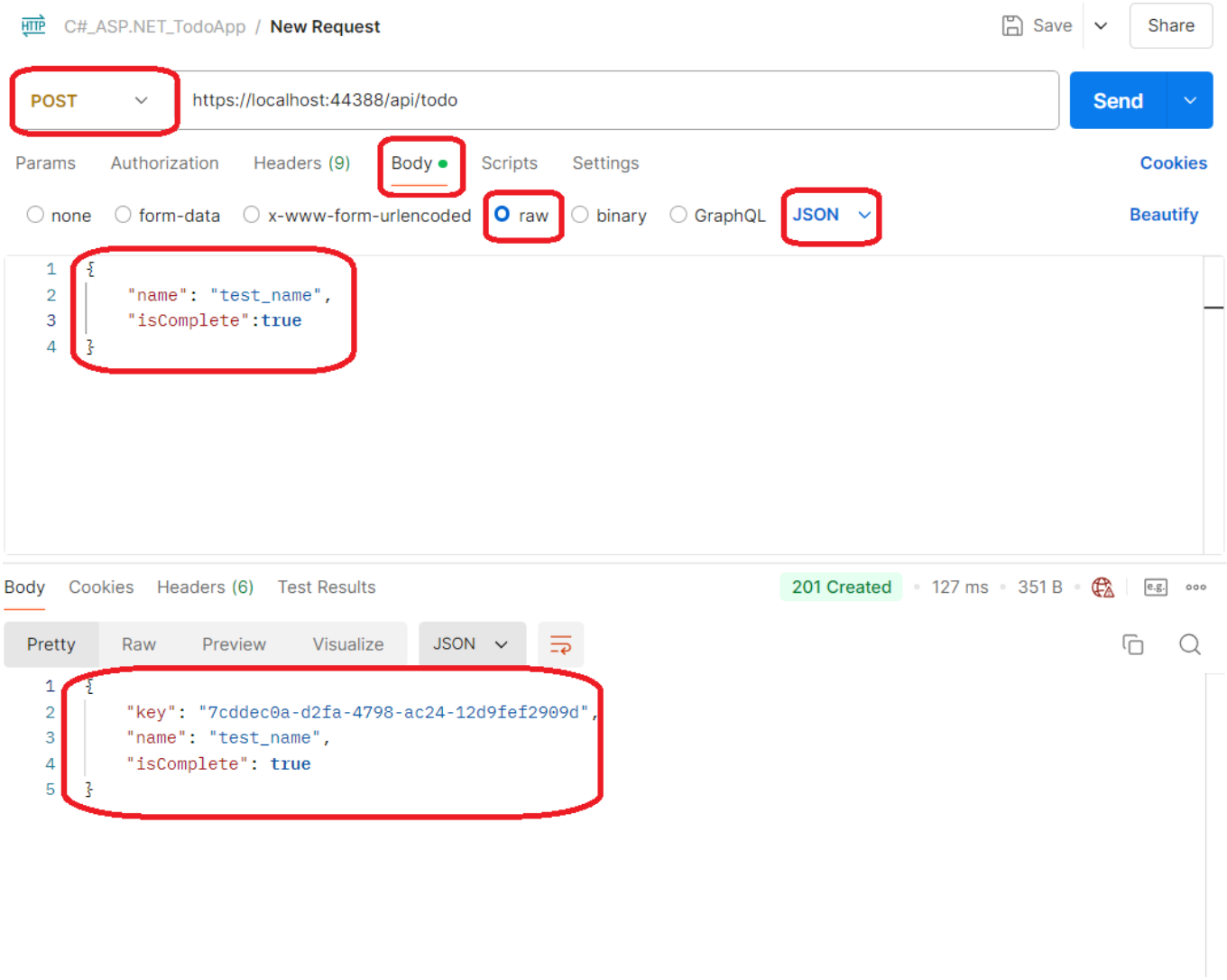
Добавление

```
[HttpPost]
public IActionResult Create([FromBody] TodoItem item)
{
    if (item == null)
    {
        return BadRequest();
    }
    TodoItems.Add(item);
    return CreatedAtRoute("GetTodo", new { id = item.Key }, item);
}
```

Это POST метод HTTP, обозначаемый атрибутом **[HttpPost]**. Атрибут **[FromBody]** сообщает MVC, что нужно получить значение элемента to-do из тела HTTP-запроса.

Метод **CreatedAtRoute** возвращает ответ 201, который является стандартным ответом для метода HTTP POST, создающего новый ресурс на сервере. **CreateAtRoute** также в ответ добавляется заголовок Location . В заголовке Location указывается URI вновь созданного элемента to-do.

Используйте Postman для отправки запроса на создание



- Установите для метода HTTP значение **POST**
- Нажмите переключатель **Body**
- Нажмите переключатель **raw**
- Установите тип JSON
- В редакторе ключ-значение введите элемент задачи, например

```
{
    "name":"<your to-do item>"
}
```

- Нажмите **Отправить**

Перейдите на вкладку Заголовки и скопируйте заголовок **Location**:

C#_ASP.NET_TodoApp / New Request

Save Share

POST https://localhost:44388/api/todo Send

Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
1 {
2     "name": "test_name",
3     "isComplete":true
4 }
```

Body Cookies Headers (6) Test Results 201 Created 60 ms 351 B

Key	Value
Transfer-Encoding	chunked
Content-Type	application/json; charset=utf-8
Location	https://localhost:44388/api/Todo/dc799a16-c1f6-49b1-9879-09ba11776c62
Server	Microsoft-IIS/10.0
X-Powered-By	ASP.NET
Date	Tue, 05 Nov 2024 10:59:24 GMT

Вы можете использовать URI заголовка Location для доступа к только что созданному ресурсу. Метод **GetById** создал именованный маршрут **"GetTodo"**:

```
[HttpGet("{id}", Name = "GetTodo")]
public IActionResult GetById(string id)
```

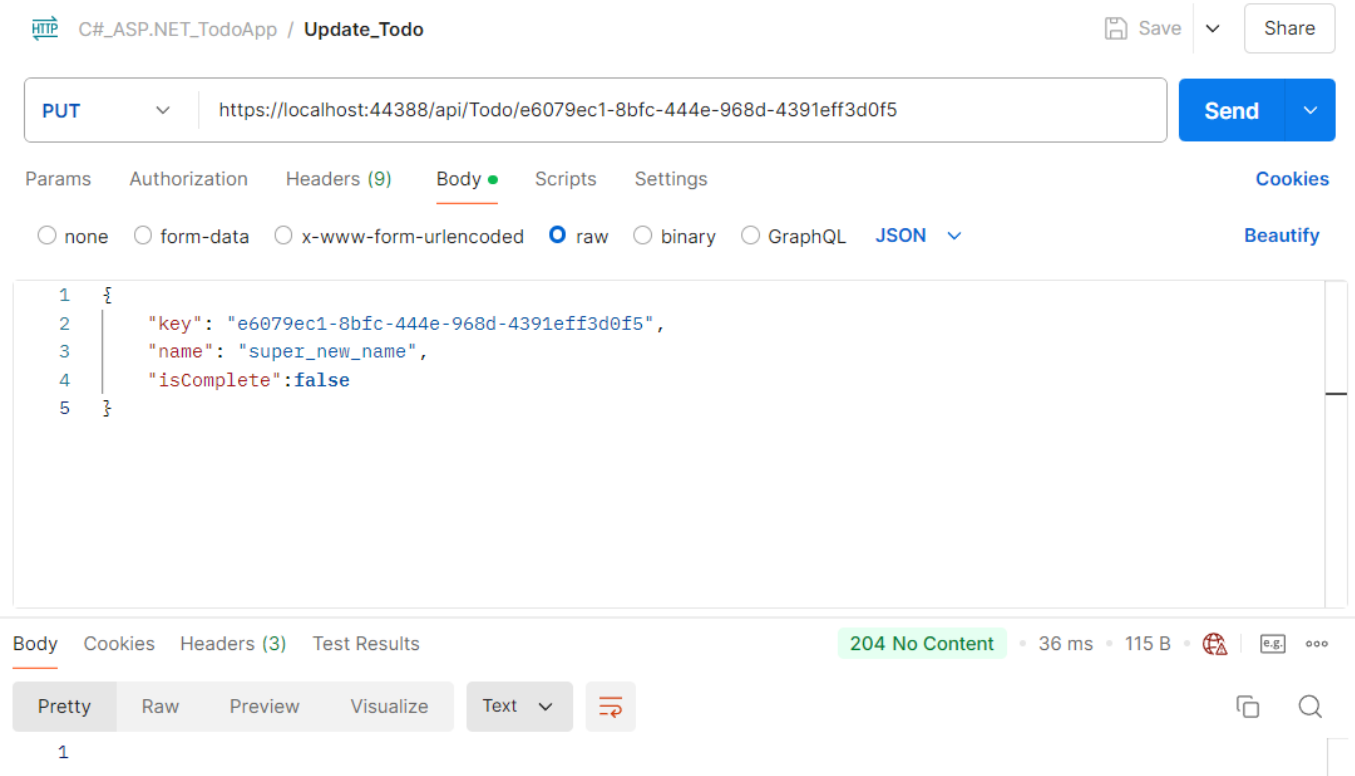
Обновление

```
[HttpPut("{id}")]
public IActionResult Update(string id, [FromBody] TodoItem item)
{
    if (item == null || item.Key != id)
    {
        return BadRequest();
    }
```

```
    var todo = TodoItems.Find(id);
    if (todo == null)
    {
        return NotFound();
    }

    TodoItems.Update(item);
    return new NoContentResult();
}
```

Update похож на **Create**, но использует метод HTTP PUT. Ответ - 204 (без содержимого). Согласно спецификации HTTP, PUT-запрос требует, чтобы клиент отправил весь обновленный объект, а не только измененные данные. Для поддержки частичных обновлений используется HTTP PATCH.



Обновление с Patch

Данная перегрузка похожа на показанную ранее **Update**, но использует HTTP- метод Patch.

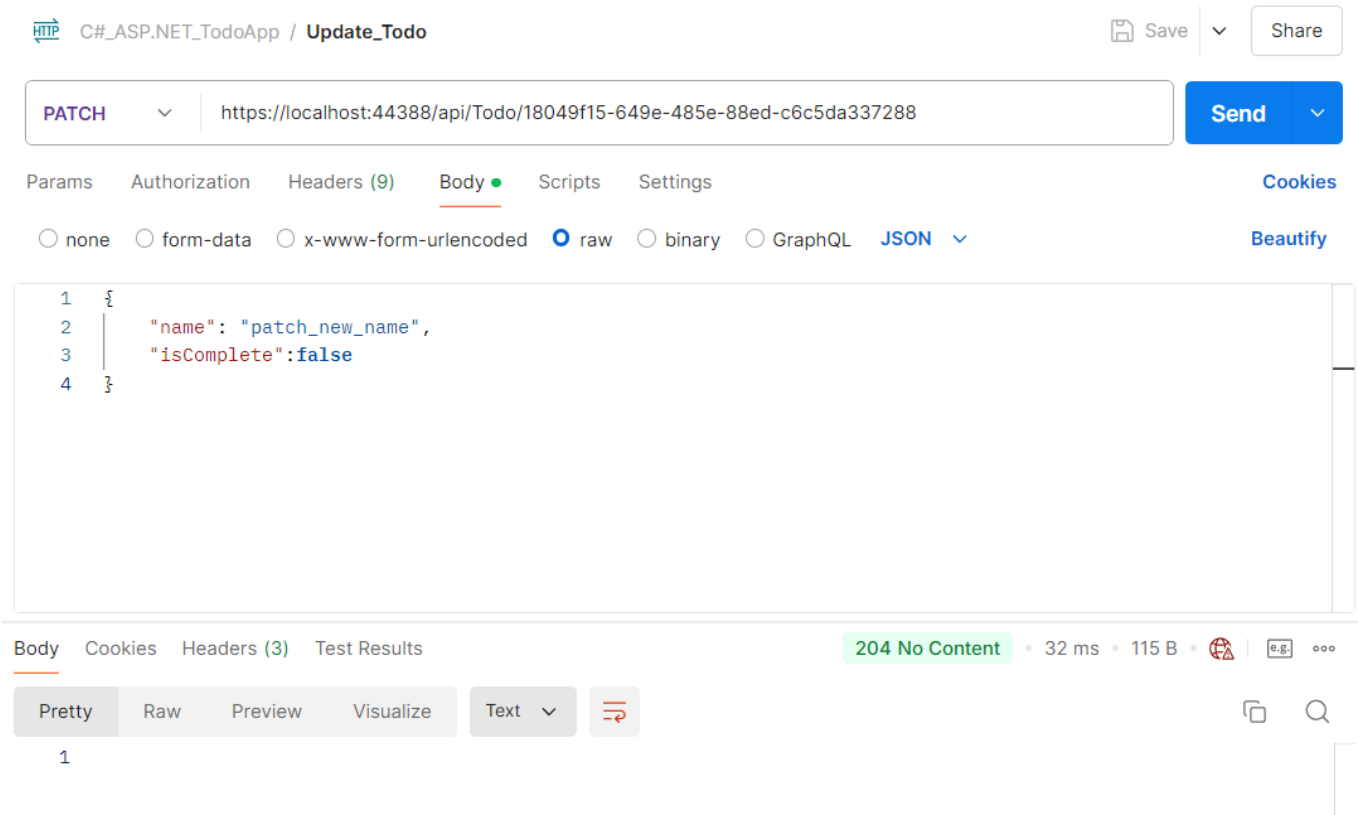
```
[HttpPatch("{id}")]
public IActionResult Update([FromBody] TodoItem item, string id)
{
    if (item == null)
    {
```

```
        return BadRequest();
    }

    var todo = TodoItems.Find(id);
    if (todo == null)
    {
        return NotFound();
    }

    item.Key = todo.Key;

    TodoItems.Update(item);
    return new NoContentResult();
}
```



Удаление

```
[HttpDelete("{id}")]
public IActionResult Delete(string id)
{
    var todo = TodoItems.Find(id);
    if (todo == null)
    {
        return NotFound();
    }
```

```
    TodoItems.Remove(id);
    return new NoContentResult();
}
```

C#_ASP.NET_TodoApp / **Delete_Todo**

Save | Share

DELETE https://localhost:44388/api/Todo/61c318a0-a9b7-4c8e-ad87-737e88caa7f5 **Send**

Params | Authorization | Headers (7) | Body | Scripts | Settings | Cookies

Query Params

	Key	Value	Description	Bulk Edit
	Key	Value	Description	

Body | Cookies | Headers (3) | Test Results

204 No Content • 25 ms • 115 B

Pretty | Raw | Preview | Visualize | Text

```
1
```